Package 'tripletTools'

May 16, 2026

Type Package

Title Tools for working with triadic comparisons data

Version 0.2.0

Description This package contains a variety of functions for loading, processing and analyzing data generated by triadic comparisons studies.

License MIT + file LICENSE

Encoding UTF-8

LazyData true

RoxygenNote 7.3.2

Suggests knitr,
rmarkdown,
reticulate,
testthat (>= 3.0.0)

Config/testthat/edition 3

Roxygen list(markdown = TRUE)

Depends R (>= 3.5)

Imports ape,
png,
vegan,
data.table,
dplyr,
magrittr,
readr,
stringr,
tools,
rlang

VignetteBuilder knitr

URL https://knowledge-and-concepts-lab.github.io/tripletTools/

Contents

`align.embeddings` *Align embeddings across participants*

Description

This function takes a list of embeddings, one per participant, and procrustes-aligns each to a reference embedding.

Usage

```
align.embeddings(emb, scl = TRUE, baseno = 1)
```

Arguments

`emb`	List of embeddings; each element is one participant.
`scl`	Flag indicating whether alignment should allow scaling
`baseno`	Integer indicating which list element to use as the base.

Details

Each element of the list should contain embedding data from one participant, as returned by the function `get.combined`. Each embedding must contain the same items in the same order, and must be of the same dimension. The function will procrustes-align each participant's embedding to the base participant, specified by the `baseno` argument. The procrustes alignment permits rotation, translation, scaling and reflection.

Value

A list with each participant's embedding aligned to the base embedding.

Examples

```
#Example data for subject 1
s1 <- data.frame(
      x = c(1:10) + runif(10)/10,
      y = c(1:10) + runif(10)/10
      )

s2 <- s1 * 2 + 4 #Double and shift s1 data
s2[,1] <- -1 * s2[,1] #Reflect along one dimension

unaligned <- list(s1,s2)
aligned <- align.embeddings(unaligned)
```

`assign_sample_alg` *Label each trial with its sampling algorithm*

Description

Adds a `sampleAlg` column to a trial-level data frame based on the structure of each row. This is used with legacy data that does not include a `trial_category` column exported directly from jsPsych. For data collected with the current experiment template, `trial_category` is available directly and can be renamed to `sampleAlg` instead.

Usage

```
assign_sample_alg(d)
```

Arguments

`d`	Data frame with columns `head`, `winner`, `loser`, and `validation`.

Details

Classification rules (applied in order):

1. If `head == winner` or `head == loser`, the trial is a catch (attention check) trial: `"check"`.
2. If `validation == TRUE`, the trial is a validation trial: `"validation"`.
3. Otherwise: `"random"`.

Value

The input data frame with an additional character column `sampleAlg` containing `"check"`, `"validation"`, or `"random"`.

Examples

```
## Not run:
d <- data.frame(
  head       = c("cat", "dog", "bird"),
  winner     = c("cat", "fish", "eagle"),
  loser      = c("dog", "frog", "sparrow"),
  validation = c(FALSE, FALSE, TRUE)
)
assign_sample_alg(d)

## End(Not run)
```

---

assign_sample_sets *Assign trials to train or test sets*

Description

Randomly assigns each non-catch trial to either a training set or a test set using a stratified split within each participant. Catch trials (`sampleAlg == "check"`) receive `NA` and are excluded from both sets.

Usage

```
assign_sample_sets(df, test_prop = 0.2, seed = 42)
```

Arguments

`df`	Data frame with columns `worker_id` and `sampleAlg`. `sampleAlg` must contain the values `"check"`, `"random"`, and/or `"validation"`.
`test_prop`	Numeric between 0 and 1. Proportion of non-check trials to assign to the test set. Default: `0.2`.
`seed`	Integer. Random seed passed to `set.seed` for reproducibility. Default: `42`.

Details

The split is performed per participant so that each participant contributes approximately `test_prop` of their trials to the test set. Setting `seed` ensures the assignment is reproducible.

Value

The input data frame with an additional character column `sampleSet` containing `"train"`, `"test"`, or `NA` (for catch trials).

Examples

```
## Not run:
d <- data.frame(
  worker_id = rep("p1", 10),
  sampleAlg = c(rep("random", 8), rep("check", 2))
)
assign_sample_sets(d, test_prop = 0.2, seed = 1)

## End(Not run)
```

`filter_failed_catch` *Exclude participants who fail too many catch trials*

Description

Catch trials present the target image as one of the two choices; a correct response requires selecting the choice that matches the target. This function removes any participant whose proportion of incorrect catch-trial responses exceeds `max_prop_wrong`.

Usage

```
filter_failed_catch(d, max_prop_wrong = 0.2)
```

Arguments

`d`	Data frame with columns `worker_id`, `head`, `winner`, and `loser`.
`max_prop_wrong`	Numeric between 0 and 1. Participants with a proportion of wrong catch-trial responses strictly greater than this value are removed. Default: `0.2`.

Details

A catch trial is identified by `head == winner` (correct response) or `head == loser` (incorrect response). Only rows that satisfy one of these conditions contribute to the catch-trial counts; other trial types are ignored.

Value

The input data frame with rows belonging to excluded participants removed.

Examples

```
## Not run:
d <- data.frame(
  worker_id = rep(c("p1", "p2"), each = 5),
  head      = c("cat","cat","dog","dog","cat",  "bird","bird","fish","fish","bird"),
  winner    = c("cat","dog","cat","dog","fox",   "bird","crow","fish","tuna","crow"),
  loser     = c("dog","cat","dog","cat","bear",  "crow","bird","tuna","fish","bird")
)
filter_failed_catch(d, max_prop_wrong = 0.4)

## End(Not run)
```

filter_fast_responders *Exclude participants with suspiciously fast mean reaction times*

Description

Removes participants whose mean reaction time falls below a threshold, which is used as a heuristic for detecting random or inattentive responding. The comparison is performed on the log scale so that the threshold is applied uniformly regardless of RT distribution skew.

Usage

```
filter_fast_responders(d, min_mean_rt_ms = 1000)
```

Arguments

d	Data frame with columns worker_id and rt. rt will be coerced to numeric via as.numeric; non-numeric values become NA and are excluded from the mean.
min_mean_rt_ms	Numeric. Minimum mean RT in milliseconds. Participants with mean(rt) < min_mean_rt_ms are removed. Default: 1000.

Value

The input data frame with rows belonging to excluded participants removed.

Examples

```
## Not run:
d <- data.frame(
  worker_id = c(rep("p1", 100), rep("p2", 100)),
  rt        = c(rnorm(100, 1500, 200), rnorm(100, 300, 50))
)
filter_fast_responders(d, min_mean_rt_ms = 1000)
# only p1 is retained

## End(Not run)
```

filter_incomplete *Exclude participants with too few trials*

Description

Removes participants who completed fewer than min_trials experiment trials, as well as any rows with a missing worker_id. This guards against partially-completed sessions being included in downstream analyses.

Usage

```
filter_incomplete(d, min_trials = 510)
```

Arguments

d	Data frame with a worker_id column.
min_trials	Integer. Minimum number of trials required for a participant to be retained. Default: 510.

Value

The input data frame with rows belonging to excluded participants (and rows with NA worker IDs) removed.

Examples

```
## Not run:
d <- data.frame(
  worker_id = c(rep("p1", 600), rep("p2", 100)),
  rt        = rnorm(700, 1500, 300)
)
filter_incomplete(d, min_trials = 510)
# only p1 is retained

## End(Not run)
```

---

get.combined *Get combined data*

Description

This function reads a triplet or embedding data file containing data from multiple participants, returning a named list with each element containing data from one participant.

Usage

```
get.combined(fname, eflag = FALSE)
```

Arguments

fname	Path to and name of the data file.
eflag	Flag indicating whether data are embeddings, default FALSE

Details

Data files must be in CSV format with column names in the first line. The function assumes participant identifier labels are included in a field called worker_id. If the data are embeddings, the file must contain column names called dim_x where x is the dimension number. Embedding data must also include a column named item that indicates the item embedded at each row. The function will use this column to set row names for each dataframe. The list elements will be labelled by the worker_id value.

Use this function to read in combined (across subjects) triplet data files (with eflag=FALSE) or embedding files (with eflag=TRUE)

Value

A named list, each element being a dataframe containing one participant's data.

Examples

```
fpath <- system.file("extdata", "cfd36_embeddings_individual.csv", package="tripletTools")

embeddings <- get.combined(fpath, eflag=TRUE)

head(embeddings[[1]])
```

`get.group.list.mean` *Get group list mean*

Description

From a list of embeddings and a vector indicating to which group each embedding belongs, this function aligns embeddings within each group, then computes the mean embedding across members of the group.

Usage

```
get.group.list.mean(elist, grps)
```

Arguments

`elist`	List of embeddings.
`grps`	Vector indicating to which group each individual in `elist` belongs. Elements must be in the same order as in elist.

Details

This function is useful for visualizing the mean embedding for different groups without having to recompute it from scratch. Note that the average of the aligned embeddings returned by this function will necessarily be the same as what is found if the embeddings *are* computed from scratch from the same participants.

Value

List containing mean embedding for each group.

Examples

```
repdist <- get.rep.dist(icon_emb_ind) #Representational distances
hc <- hclust(as.dist(repdist), method = "ward.D") #Cluster tree
clusts <- cutree(hc, 2) #Cut into 2 clusters

mn.by.clust <- get.group.list.mean(icon_emb_ind, clusts)
plot_pics(mn.by.clust[[1]], icon_pics)
```

`get.hoacc` *Get hold-out prediction accuracy*

Description

This function computes embedding prediction accuracy on held-out items.

Usage

```
get.hoacc(em, td, trialtype = "test", isemb = TRUE)
```

Arguments

`em`	Embedding or distance matrix for generating predictions.
`td`	Dataframe containing triplet data to be evaluated.
`trialtype`	Type of trial to be evaluated, defaults to `test`
`isemb`	When true, `em` is a matrix of embedding coordinates; when false, it is assumed to be a distance matrix.

Details

This is a wrapper function for `test.model` that computes the predicted response for each triplet of the specified type in the triplet dataset, then compares this to the true answer and computes the proportion of trials for which the prediction is matched.

Triplet data must be in dataframe objects containing columns labeled `Center`, `Left`, `Right`, and `Answer` as well as a column labeled `sampleSet` that indicates the trial type for each triplet. Embedding data must be a numeric matrix (or coercible to one) containing either the embedding coordinates for each item or a matrix of item-to-item distances.

Value

A real-valued number indicating the proportion of trials (of the indicated type) for which the prediction was correct.

Examples

```
m <- data.frame(
  x=c(1,1.1,2,2.1),
  y=c(1.25,1.75,1.25,1.75))

row.names(m) <- c("cat","dog","car","boat")

m <- as.matrix(m)

tr <- data.frame(
   Center=c("cat", "car", "dog", "boat"),
   Left = c("dog", "boat", "car", "cat"),
   Right= c("car", "dog", "boat", "car"),
   Answer=c("dog", "boat", "car", "car"),
   sampleSet=c("test","test","test", "test"))

get.hoacc(m, tr, isemb=TRUE)
```

get.nearest.k *Get nearest k*

Description

This function takes a distance matrix and an item name and returns the k nearest neighbors to the specified item.

Usage

```
get.nearest.k(dmat, item, k = 5)
```

Arguments

dmat	Data matrix; rows must be named.
item	String indicating the item for which nearest neighbors will be returned
k	How many neighbors to return.

Details

dmat must be a matrix with named rows, and item must match the name of one row.

This function is useful for understanding local similarity structure in a higher- dimensional embedding, and for creating a k-nearest-neighbor graph of such structure.

Value

A character vector containing the k nearest neighbors in order of proximity to the target item.

Examples

```
emb <- icon_emb_ind[[1]] #Embedding for participant 1
fdists <- as.matrix(dist(emb)) #Compute pairwise distance matrix
target <- "fdfob"  #Name of target item

#Return 5 items nearest to target:
get.nearest.k(fdists, target, 5)
```

get.participant.summary *Get participant summary*

Description

This function takes a list of triplet data of the kind returned by get_combined and from it generates a dataframe summarizing information about each participant in the study.

Usage

```
get.participant.summary(
  d,
  irange = NULL,
  mintrial = 1000,
  accthresh = 0.8,
  rtthresh = 0
)
```

Arguments

d	List of triplet data. Each element is data from one participant.
irange	Vector indicating which elements of the list to include. Default is all.
mintrial	Minimum number of trials needed to count as a complete record.
accthresh	Accuracy threshould for check trials to pass quality check
rtthresh	Threshold of log RT to pass quality check

Details

The summary will include participant ID, number of completed trials, mean accuracy on check trials, and mean log(RT) across all trials. The arguments `accthresh` and `rtthresh` set criteria for assessing the participant's data quality. A mean log RT of 0 or less means participant was responding in under one second on average, usually too fast for data to be real. Chance responding will yield an accuracy of 0.5 on check trials, so a threshold of 0.8 means participant was likely guessing on at least 40 percent of trials.

This function assumes standard triplet data naming conventions for column names.

Value

Data frame containing information about each participant in the study.

Examples

```
#Path to example triplet data
fpath <- system.file("extdata", "cfd36_triplets_individual.csv", package = "tripletTools")

#Read the data
trips <- get.combined(fpath)

#Compute summary
part.summary <- get.participant.summary(trips)

head(part.summary)
```

`get.prediction.matrix` *Get prediction matrix*

Description

This function takes a list of embeddings and a matched list of triplet data and uses each embedding to predict responses on each triplet dataset, returning the mean accuracy as a matrix.

Usage

```
get.prediction.matrix(elist, tlist, ttype = "test")
```

Arguments

`elist`	Named list of embeddings.
`tlist`	Named list of triplet data.
`ttype`	Type of trial to evaluate on, defaults to test trials

Details

Elements of the embedding and triplet lists should be named with the participant ID, as in the format returned by `get_combined`. Ideally these lists will contain the same participants in the same order. The function expects files conform to naming conventions. It first extracts trials of the indicated type, then uses `get.hoacc` to compute the proportion of items for which the embedding predicts the correct response. It loops through all embeddings and datasets, returning a matrix of the corresponding prediction accuracies.

Assuming the two lists contain the same participants in the same order, the matrix diagonal will indicate how well a participant's own embedding predicts their decisions on the test items (typically hold-out items not used to compute the embedding).

Value

A matrix. Each row is an embedding, each column a triplet dataset. Entries indicate how accurately the embedding predicts the triplet data for the trial type indicated.

Examples

```
pmat <- get.prediction.matrix(icon_emb_ind, icon_triplets)
pmat
```

get.raster.from.png *Get raster from PNG*

Description

This function reads a PNG file and converts it to a raster object with a transparent background for plotting.

Usage

```
get.raster.from.png(f)
```

Arguments

f	Filename for PNG image.

Details

When plotting line-drawings as points on a scatterplot (using plot_pics for instance), it can be useful to keep the line drawing background transparent and to be able to control the color in which the image is plotted. Converting PNGs to rasters allows this functionality. This function converts the PNG to a raster object; setting the pc argument in plot_pics then controls the color when plotting the image.

get.rep.dist *Get representational distances*

Description

Given a list of embeddings, this function computes the procrustes distance between each pair and returns this as a distance matrix.

Usage

```
get.rep.dist(elist, rootflag = TRUE)
```

Arguments

elist	List of embeddings.
rootflag	Computer square root of distance? Defaults to TRUE

Details

Each element of the list should contain a matrix of embedding coordinates from one participant. Each embedding should contain the same items in the same order, and should be of the same dimension.

By default the distance metric is the procrustes equivalent of Pearon's correlation, that is `1 - sqrt(1 - ss)` where `ss` is the normalized sum of squares from the aligned embeddings. If `rootflag=FALSE`, the normalized sum of squares is used as the distance metric.

Value

A matrix of distances between each pair of embeddings.

Examples

```
#Subject 1 data
s1 <- matrix(
      c(1,1,
      2,2,
      3,3,
      4,4,
      5,5), 5,2,byrow = TRUE)

#Subject 2 is noisy version of subject 1
s2 <- s1 + runif(10) / 10

#Subject 3 is different:
s3 <- matrix(
      c(1,2,
      3,4,
      4,3,
      2,1,
      5,2), 5,2,byrow = TRUE)

slist <- list(s1,s2,s3)

sdist <- get.rep.dist(slist)

head(sdist)
```

---

get.tip.coords *Get tip coordinates*

Description

After generating a phylogram plot, this function returns the coordinates of the tree tips on the plotting surface, useful for plotting images or other symbols at the ends of the tree plot.

Usage

```
get.tip.coords()
```

Details

The `phylogram` plot type from the `ape` package invisibly stores the tip coordinates of the plot. This function retrieves them and returns them as a matrix useful for adding symbols or images to the leaves of the tree.

Value

A matrix of x, y coordinates indicating the locations of the tips of the phylogram.

Examples

```
#Make a matrix
dmat <- matrix(
        c(1, 1, 1, 0, 0, 0,
        1, 1, 1, 0, 0, 0,
        1, 1, 1, 0, 0, 0,
        0, 0, 0, 1, 1, 1,
        0, 0, 0, 1, 1, 1,
        0, 0, 0, 1, 1, 1
        ),6,6)

#Noise it up
dmat <- dmat + matrix(runif(36)/2,6,6)

#Compute a cluster analysis
hc <- stats::hclust(stats::dist(dmat))
pt <- ape::as.phylo(hc)

#Cut tree to give two clusters
clusts <- stats::cutree(hc, 2)

#Make plot with no tip labels
plot(pt, show.tip.label = FALSE)

#Add colored points to tips
points(get.tip.coords(), pch = 16, col = c("orange", "blue")[clusts])
```

---

icon_emb_group *Group embedding data for 32 icon images of faces and buildings*

Description

This dataset contains embedding coordinates from a triplet study using 32 icon images showing faces and buildings. All stimuli vary in age (old/young) and time (day/night). Faces also vary in gender and race; places vary in size and kind (house/church). Five participants were asked to judge which option was more similar to the referent without further instruction. The object is a single data frame containing 3d embedding coordinatwes computed from the training trials for all participants.

Usage

```
icon_emb_group
```

Format

`icon_emb_group`**:**

A data frame with 32 rows (items) and columns as follows:

dim_0, dim_1, dim_2 First, second and third dimensions of the embedding.

worker_id Set to group since this is a group embedding

item Name of the stimulus item at that embedding location.

path path to stimulus file

Details

The letters in the stimulus identifier indicate features of the corresponding icon as follows: face/place, day/night, female/male/church/house, old/young, black/white/big/small.

Source

Colon et al., in preparation.

`icon_emb_ind` *Individual embedding data for 32 icon images of faces and buildings*

Description

This dataset contains embedding coordinates from a triplet study using 32 icon images showing faces and buildings. All stimuli vary in age (old/young) and time (day/night). Faces also vary in gender and race; places vary in size and kind (house/church). Five participants were asked to judge which option was more similar to the referent without further instruction. Three-D embeddings were computed separately for each participant.

Usage

```
icon_emb_ind
```

Format

`icon_emb_ind`**:**

A list with five elements, each containing an embedding from one person. The embedding is a data frame object with 32 rows (items) and six columns as follows:

dim_0, dim_1, dim_2 First, second and third dimensions of the embedding.

worker_id Random code identifying each participant

item Name of the stimulus item at that embedding location.

path path to stimulus file

Details

The letters in the stimulus identifier indicate features of the corresponding icon as follows: face/place, day/night, female/male/church/house, old/young, black/white/big/small.

Source

Colon et al., in preparation.

icon_pics *Face and Place icon pictures*

Description

This dataset contains PNG format images of 32 images showing icons of faces and buildings. These are the stimuli used in the demo experiment included as part of this package.

Usage

```
icon_pics
```

Format

`icon_pics`:
A named list with 32 elements, each containing one image.

Details

The letters in the stimulus identifier indicate features of the corresponding icon as follows: face/place, day/night, female/male/church/house, old/young, black/white/big/small.

Source

https://www.chicagofaces.org/

icon_triplets *Triplet data for 32 icons of faces and places*

Description

A list containing triplet judgments for 5 participants on 32 icons showing faces and buildings. These triplets were used to compute embeddings of the 32 items separately for each participant (`icon_emb_ind`) as well as a single group embedding (`icon_emb_group`).

Usage

```
icon_triplets
```

Format

`icon_triplets`:
A named list, each containing a dataframe with 11 columns:

head, winner, loser Integer indices for items in a given triplet.
worker_id Random identifier for each participant.
rt Response time on triplet (in miliseconds).
Center The target item.
Left, Right The option items appearing on the left and right.
Answer The option item chosen by the participant.
sampleAlg The algorithm used to sample the item.
sampleSet Which set the sampled item belongs to.

Details

Each element of the list contains the triplet data for one participant in the study. Rows of a dataframe correspond to a single trial. The elements of the full list are named by the random participant ID number.

Each triplet can be sampled in one of four ways indicated by sampleAlg:

1. *random*: Sampled randomly with uniform probability from all triplets.
2. *validation*: Sampled randomly from a fixed, pre-specified set of possible triplets.
3. *check*: Sampled from a small set of items where the answer is obvious, used to check attention and data quality.
4. *adaptive*: Sampled according to some adaptive sampling algorithm.

The column `sampleSet` indicates how the triplet is to be used in computing and evaluating embeddings:

1. *train*: Triplets used to fit embedding.
2. *test*: Triplets held out from training, used to evaluate embedding quality.
3. NA: Triplets used for checking attention / data quality.

Validation trials can be used to measure the extent of inter-subject agreement: since these are drawn from a common pool, these triplets will appear repeatedly across participants. Thus for each such triplet one can compute the majority vote across participants who received the triplet, and the proportion of those participants who agree with the majority vote. This gives an estimate of how consistent judgments are across participants.

Source

Colon et al., in prep

`make.tripnames` *Make triplet names*

Description

This function creates a unique name for each triplet appearing in a triplet data frame.

Usage

```
make.tripnames(tripdat)
```

Arguments

`tripdat`	A data frame containing triplet data; must conform to naming conventions.

Details

This function is useful for finding responses to a given triplet, which is especially important when computing within and between-participant consistency on validation trials.

Value

A character vector containing the unique triplet name for each trial in the triplet dataframe.

Examples

```
trips <- icon_triplets[[1]] #Triplet data for participant 1
tnames <- make.tripnames(trips) #Make triplet names
tnames[1:5] #Names of first five triplets
```

`make.vmat` *Make validation matrix*

Description

This function uses the validation trials from triplet data to construct a subject-by-trial matrix of judgments on these items.

Usage

```
make.vmat(triplist)
```

Arguments

`triplist`	Named list whose elements each contain triplet data from one participant. Names should be participant identifiers, as returned by `get_combined`.

Details

Each validation triplet is identified with a unique code of the kind generated by `make.tripnames` which indicates the target word and the two options, with the options ordered alphabetically.

Where a participant judged a particular validation triplet the entries of `bysbj` will indicate the proportion of times the participant's decision agreed with the majority vote. (In most cases this will be 0 or 1 since participants usually judge a validation triplet only once; however in some studies the same validation trials are repeated to evaluate self-consistency). Where a participant did not judge a given triplet, the entry in `bysbj`will be NA.

Value

Names list with two elements. `majority` is a matrix with one row per validation triplet, reporting the winning choice and proportion of participants who selected ite. `bysbj` is a matrix indicating, for each participant (rows) and validation triplet (columns), the choice the participant made.

Examples

```
vmat <- make.vmat(icon_triplets)
vmat$majority
```

`model.strength` *Get model strength*

Description

This function assesses how well the similarities expressed in an embedding or distance model explain the triad choices contained in vdat by computing the distance of the furthest option divided by the sum of both distances.

Usage

```
model.strength(model, vdat, isemb = TRUE)
```

Arguments

`model`	An embedding model.
`vdat`	A matrix of triplet data.
`isemb`	Flag indicating whether model is an embedding; if FALSE, model is treated as a distance matrix.

Details

If `isemb==FALSE` function assumes `model` is a distance matrix.

The prediction-strength metric has a floor of 0.5 (both equally distant) and increases to approach 1.0 when one option is very close and the other very far, that is, when the answer should be obvious. If the embedding is good, people should make reliably consistent decisions for triplets where this 'strength' is high and less consistent decisions when it is low.

This measure can be especially useful when computed for validation trials where the proportion of participants agreeing with the majority vote on a trial has been computed. If most participants agree with the majority decision on the triplet, this means decisions are highly reliable across participants. If the embedding is good, such items will also have a high prediction-strength score. Thus the correlation between prediction strength and inter-subject agreement tells you something about the quality of the embedding.

Value

Vector containing the prediction strength metric for each triplet in `vdat`

Examples

```
emb <- icon_emb_ind[[1]] #Embedding for participant 1
trips <- icon_triplets[[1]] #Triplets for participant 1

#Get validation trials only
vdat <- subset(trips, trips$sampleAlg=="validation")

pstrength <- model.strength(emb, vdat)
```

pacc.by.cluster *Prediction accuracy by cluster*

Description

This function computes how well a participant's held-out judgments are predicted by their own embedding and embeddings from members of the same or other clusters.

Usage

```
pacc.by.cluster(pacc, clusts, samediff = TRUE)
```

Arguments

pacc	Participant-by-participant matrix of predictive accuracies of the kind returned by get_prediction_matrix.
clusts	Vector indicating the cluster membership for each participant in pacc
samediff	If TRUE, returns mean prediction accuracy for participants in same cluster vs mean from those in different cluster. Otherwise returns mean prediction accuracy from participants in each cluster.

Details

Participants can be clustered based on their pairwise representational distances as returned by get.rep.dist. This function will then compute how well, on average, embeddings within a cluster predict each participant's held-out (test) judgments. It also returns how well the participant's own embedding predicts their held-out judgments. If clusters are useful, the participant's judgments should be better-predicted by their cluster-mates than by non-cluster-mates. If cluster-mates all share the same embedding, same-cluster prediction should be as good a own-embedding prediction.

The first column of the returned matrix is always prediction accuracy from the participant's own embedding. If samediff==TRUE the returned matrix will include mean prediction accuracy from the participant's own cluster and from all participants not in the same cluster. If FALSE, the returned matrix will include one column per cluster, and entries will indicate mean prediction accuracy from the embeddings in that cluster.

Value

A participant-by-cluster matrix indicating the mean accuracy predicting each participant's judgments from embeddings in each cluster.

Examples

```
repdist <- get.rep.dist(icon_emb_ind) #Representational distances

#Hierarchical cluster
hc <- hclust(as.dist(repdist), method = "ward.D")
clusts <- cutree(hc, 2) #Cut tree to yield two clusters

pacc <- get.prediction.matrix(icon_emb_ind, icon_triplets) #Prediction matrix
pbc <- pacc.by.cluster(pacc, clusts, samediff=TRUE)

colMeans(pbc)
```

plot_cis *Plot column means and confidence intervals*

Description

Plot column means and confidence intervals

Usage

```
plot_cis(
  d,
  barflag = FALSE,
  newflag = TRUE,
  xvals = NULL,
  rgbvec = c(0, 0, 1),
  ...
)
```

Arguments

d	A matrix of numerical data
barflag	Flag indicating whether a barplot is desired
newflag	Should new plot be generated, default T
xvals	Vector of x values equal in length to number of columns in d
rgbvec	Vector of red, green, blue proportions
...	Other graphical parameters

Details

This function computes the mean and 95% confidence intervals of each column in d and plots these either as a line plot with a transparent ribbon (default) or as a barplot.

Value

Invisibly returns matrix of column means and 95% confidence intervals

Examples

```
x <- matrix(c(1:12),3,4)
plot_cis(x)
```

plot_pics *Plot pictures in a scatterplot*

Description

This function plots a list of PNG or raster images as the points in a scatterplot.

Usage

```
plot_pics(
  md,
  plist,
  x = 1,
  y = 2,
  pr = 1,
  pc = NULL,
  psize = 0.05,
  newplot = TRUE,
  ...
)
```

Arguments

md	Matrix of data indicating where each image is to be plotted.
plist	List of PNG or raster images
x	Which column of the matrix should be used for x-axis position
y	Which column of the matrix should be used for y-axis position
pr	Proportion of items to be plotted OR vector indicating which items should be plotted.
pc	If images are rasters, what color should they be plotted in?
psize	Plot size for each image as proportion of plotting surface
newplot	Should a new plot be generated? If FALSE images added to current plot.
...	Other graphical parameters

Details

The number of rows in the data matrix must be equal to the length of the list containing the images to be plotted. Each row corresponds to one image; thus the first image in the list will be plotted at the coordinates in row 1 of the data matrix.

When `newplot=FALSE` the images will be added as points to the existing plot. When used with `get.tip.coords` this can add images to the tips of phylogram plots, an effective way to visualize structure in embeddings with more than two or three dimensions.

When there are many images, they often clutter the plot, making it hard to see structure. You can control the proportion of images shown by setting `pr` to a value smaller than 1.0. In this case, a random sample of impages will be plotted.

If images are line-drawings and are loaded as rasters rather than PNGs, setting the plot color `pc` will control the color the image is displayed in. This can be specified as a single value for all images or as a vector of colors; if a vector each element sets the plot color for one image. This can be useful for displaying other data in the plot, such as the gender of the participant who produced the drawing.

Value

Generates a plot or adds images to existing plot.

Examples

```
#Plot a 2D embedding as a scatterplot using images instead of points:

emb <- icon_emb_ind[[1]] #Get embedding from first participant

plot_pics(emb, icon_pics, psize = 0.03)

#Plot the same data as a hierarchical cluster plot with images as leaves.
hc <- stats::hclust(dist(emb), method = "ward.D")
pt <- ape::as.phylo(hc)

plot(pt, type = "fan", show.tip.label=FALSE)
tip_coords <- get.tip.coords()
plot_pics(tip_coords, icon_pics, newplot = FALSE)
```

---

process_choices *Clean raw choice strings from jsPsych CSV output*

Description

Strips file-path prefixes, surrounding brackets, quotes, and the file extension from the `choices` column as exported by jsPsych. The result is a plain comma-separated string of bare stimulus names suitable for downstream splitting and matching.

Usage

```
process_choices(choices, extension = ".png")
```

Arguments

`choices`	Character vector of raw choice strings, e.g. `"[\"assets/stimuli/cat.png\",\"assets/stimuli/`
`extension`	Character. File extension to strip, including the leading dot. Default: `".png"`.

Value

Character vector of the same length as `choices`, with brackets, quotes, path prefixes, and the extension removed.

Examples

```
raw <- c(
  "[\"assets/stimuli/cat.png\",\"assets/stimuli/dog.png\"]",
  "[\"resources/apple.png\",\"resources/banana.png\"]"
)
process_choices(raw)
# [1] "cat,dog"   "apple,banana"
```

`read_legacy` *Standardise a legacy triadic comparison dataset*

Description

Reads a single CSV file from a legacy triplet experiment and converts it to the standard column format used by this package. Unrecognised or missing columns are derived where possible from the data that are present; see the **Column mappings** section below for full details. Two output files are written alongside the input: a standardised data CSV and a stimulus-level mapping CSV.

Usage

```
read_legacy(input_file)
```

Arguments

`input_file`	Character. Path to the input CSV file.

Value

A list returned invisibly with two elements:

`data_file` Path of the written standardised data CSV (input filename with `_v2025.csv` suffix).

`levels_file` Path of the written stimulus-level mapping CSV (input filename with `_2025_levels.csv` suffix).

Output columns

`p_id` Character participant identifier.

`Center` Character label of the target (reference) stimulus.

`Left` Character label of the left-hand choice stimulus.

`Right` Character label of the right-hand choice stimulus.

`Answer` Character label of the chosen stimulus.

`head` Integer (0-indexed) factor code for `Center`, ordered alphabetically across all unique stimuli in `Center`, `Left`, and `Right`.

`winner` Integer factor code for `Answer`, using the same ordering as `head`.

`loser` Integer factor code for the unchosen option, using the same ordering as `head`.

`RT` Numeric reaction time. `NA` if not present in the input.

`sampleAlg` Character sampling algorithm label: `"random"`, `"check"`, `"validation"`, `"uncertainty"`, or `NA`.

`sampleSet` Character set label: `"train"` or `"test"`.

Column mappings

Input column names are matched case-insensitively and ignoring punctuation.

Participant ID (`p_id`) Recognised input names: `sessionID`, `session_ID`, `puid`, `Participant.ID`, `worker_id`, `sub_id`, `pid`. If none are found, participants are assigned sequential IDs (`"P1"`, `"P2"`, ...).

Center Recognised input names: `Center`, `Target`.

Left Recognised input names: `Left`, `Option1`.

Right Recognised input names: `Right`, `Option2`.

winner / loser Recognised input names: `winner` / `primary` and `loser` / `alternate`. Derived from `Answer`, `Left`, and `Right` when absent.

sampleAlg Recognised input names: `sampleAlg`, `AlgSample`. Values are recoded: `"Random"` → `"random"`; `"Test"` → `"validation"`; `"check"` → `"check"`; `"uncertainty"` → `"uncertainty"`. Filled with `NA` when absent.

sampleSet Recognised input names: `sampleSet`, `Alg.Label`, `TrnTest`. When absent, 10\ randomly assigned to `"test"` and the remainder to `"train"` (seed `2025`).

Examples

```
## Not run:
result <- clean_triadic_comparisons("data/triplets.csv")
print(paste("Wrote:", result$data_file, "and", result$levels_file))

## End(Not run)
```

---

read_raw_data *Clean raw jsPsych triplet experiment data*

Description

Reads all CSV files exported from a jsPsych triplet experiment, filters to experiment trials, applies quality-control exclusions, and returns cleaned data ready for modelling. Optionally writes the results to disk.

Usage

```
read_raw_data(
  data_dir = ".",
  output_df = NULL,
  output_levels = NULL,
  min_trials = 0,
  min_mean_rt_ms = 0,
  max_prop_wrong = 1,
  test_prop = 0.1,
  seed = 42,
  stimuli_extension = ".png"
)
```

Arguments

data_dir	Character. Path to the directory containing raw CSV exports. Default: "experiment/raw_data/".
output_df	Character or NULL. File path for the cleaned trial-level CSV. Set to NULL to skip writing. Default: "icon_fp_clean.csv" (written to the working directory).
output_levels	Character or NULL. File path for the stimulus-level mapping CSV. Set to NULL to skip writing. Default: "icon_fp_levels.csv" (written to the working directory).
min_trials	Integer. Minimum number of experiment trials a participant must have completed to be retained. Passed to filter_incomplete. Default: 200.
min_mean_rt_ms	Numeric. Minimum mean reaction time in milliseconds for a participant to be retained. Passed to filter_fast_responders. Default: 200.
max_prop_wrong	Numeric between 0 and 1. Maximum proportion of failed catch trials before a participant is excluded. Passed to filter_failed_catch. Default: 0.2.
test_prop	Numeric between 0 and 1. Proportion of non-check trials assigned to the test set in the train/test split. Passed to assign_sample_sets. Default: 0.2.
seed	Integer. Random seed for reproducible train/test splitting. Passed to assign_sample_sets. Default: 42.
stimuli_extension	Character. File extension (including the leading dot) to strip from stimulus and choice names. Default: ".png".

Value

A list returned invisibly with two elements:

trials Data frame of cleaned trial-level data with columns head, winner, loser, worker_id, rt, Center, Left, Right, Answer, sampleAlg, and sampleSet.

levels Data frame mapping integer stimulus indices to file names and paths.

---

run_embeddings *Run the full embedding pipeline for all workers*

Description

Reads triplet comparison data from input_file, trains a separate embedding model with early stopping for each worker, and then trains a combined group-level embedding across all workers. Output CSV files are written to output_dir and the results are also returned as R data frames.

Usage

```
run_embeddings(
  input_file,
  additional_data_file,
  output_dir,
  d = 5L,
  max_epochs = 50000L,
  tolerance = 1e-04,
  tol_window = 10000L,
  seed = 222L
)
```

Arguments

input_file	Path to the CSV file containing all triplets (see *Input format* above).
additional_data_file	Path to a CSV file with item metadata to append to the embedding output (e.g. image filenames listed in alphabetical order). The number of rows should match the number of unique items.
output_dir	Path to the directory where output CSV files will be saved. Created automatically if it does not already exist.
d	Number of embedding dimensions. Default 5.
max_epochs	Maximum number of training epochs. Default 50000.
tolerance	Loss tolerance for early stopping. Default 1e-4.
tol_window	Epochs without improvement before early stopping triggers. Default 10000.
seed	Integer random seed for reproducibility. Default 222.

Details

For a higher-level interface that accepts triplet data already loaded into R as a named list (the format returned by get.combined), see run_embeddings_from_list.

Value

A named list with two elements:

history Data frame with one row per worker (plus one for the group model) containing: worker_id, lowest_loss, epoch, counter_from_last_update, n_train_triplets, n_test_triplets.

embeddings Data frame of all embeddings concatenated, with dimension columns (dim_0, dim_1, ...), a worker_id column, and any columns from additional_data_file.

Input format

input_file must be a CSV with at least the following columns:

worker_id Identifier for the respondent.

head Zero-based integer index of the reference item.

winner Zero-based integer index of the item judged closer to head.

loser Zero-based integer index of the item judged further from head.

sampleSet Either "train" or "test", used to split data for early stopping.

Output files

Three CSV files are written to output_dir:

model_history.csv Training history: loss, stopping epoch, and triplet counts for each worker.

embeddings_group.csv Group-level embedding only.

embeddings.csv All per-worker and group-level embeddings concatenated.

Examples

```
## Not run:
results <- run_embeddings(
  input_file           = "triplets.csv",
  additional_data_file = "item_labels.csv",
  output_dir           = "embeddings_output",
  d                    = 5L,
  max_epochs           = 50000L
)

head(results$history)
head(results$embeddings)

## End(Not run)
```

`run_embeddings_from_list` *Run the embedding pipeline from a triplet data list*

Description

A convenience wrapper around `run_embeddings` that accepts triplet data already loaded into R as a named list — the format returned by `get.combined` — rather than reading from CSV files.

Usage

```
run_embeddings_from_list(
  triplet_list,
  output_dir,
  d = 5L,
  max_epochs = 50000L,
  tolerance = 1e-04,
  tol_window = 10000L,
  seed = 222L
)
```

Arguments

`triplet_list`	A named list of data frames, one per participant, as returned by `get.combined`. Each data frame must contain the columns `worker_id`, `Center`, `Left`, `Right`, `Answer`, `sampleAlg`, and `sampleSet`.
`output_dir`	Path to the directory where output CSV files will be saved. Created automatically if it does not already exist.
`d`	Number of embedding dimensions. Default `5`.
`max_epochs`	Maximum number of training epochs. Default `50000`.
`tolerance`	Loss tolerance for early stopping. Default `1e-4`.
`tol_window`	Epochs without improvement before early stopping triggers. Default `10000`.
`seed`	Integer random seed for reproducibility. Default `222`.

Details

The function converts items to consistent zero-based integer indices (sorted alphabetically), writes temporary CSV files, calls the Python embedding pipeline, and returns results in the standard `tripletTools` format.

Value

A named list with three elements:

`individual` Named list of numeric matrices, one per participant. Each matrix has one row per item (with item names as row names) and `d` columns (`dim_0`, `dim_1`, ...).

`group` Numeric matrix of the group-level embedding, with item names as row names and `d` columns.

`history` Data frame with one row per worker (plus `"group"`) containing training diagnostics: `worker_id`, `lowest_loss`, `epoch`, `counter_from_last_update`, `n_train_triplets`, `n_test_triplets`.

Item indexing

All unique item names appearing in the `Center`, `Left`, and `Right` columns across all participants are collected and sorted alphabetically. Each item's zero-based index in this sorted list is used as the integer index for the Python model. The same ordering is applied to all participants so that indices are consistent across workers.

Filtering

Trials with `sampleAlg == "check"` are excluded before fitting the embedding (these are attention-check trials that do not reflect genuine similarity judgments). The `sampleSet` column (indicating `"train"` or `"test"`) must be present and is passed through unchanged.

Examples

```
## Not run:
results <- run_embeddings_from_list(
  triplet_list = icon_triplets,
  output_dir   = "embeddings_output",
  d            = 3L,
  max_epochs   = 50000L
)

# Group embedding
head(results$group)

# First participant's individual embedding
head(results$individual[[1]])

# Training diagnostics
results$history

## End(Not run)
```

`run_group_embedding_from_list` *Compute a group-level embedding from a triplet data list*

Description

Trains a single embedding on the combined triplet judgments from all participants. Individual per-participant embeddings are not computed. Use this function when you only need a group summary of the similarity structure, which is faster than `run_embeddings_from_list`.

Usage

```
run_group_embedding_from_list(
  triplet_list,
  d = 5L,
  max_epochs = 50000L,
  tolerance = 1e-04,
  tol_window = 10000L,
  seed = 222L
)
```

Arguments

`triplet_list`	A named list of data frames, one per participant, as returned by `get.combined`. Each data frame must contain the columns `Center`, `Left`, `Right`, `Answer`, `sampleAlg`, and `sampleSet`.
`d`	Number of embedding dimensions. Default `5`.
`max_epochs`	Maximum number of training epochs. Default `50000`.
`tolerance`	Loss tolerance for early stopping. Default `1e-4`.
`tol_window`	Epochs without improvement before early stopping triggers. Default `10000`.
`seed`	Integer random seed for reproducibility. Default `222`.

Value

A named list with three elements:

`embedding` Numeric matrix with one row per item (item names as row names) and `d` columns (`dim_0`, `dim_1`, ...).

`loss` Best test loss achieved during training.

`history` Data frame with one row per epoch and columns `epoch`, `train_loss`, `test_loss`, `train_acc`, `test_acc`.

Item indexing

All unique item names appearing in the `Center`, `Left`, and `Right` columns across all participants are collected and sorted alphabetically. Each item's zero-based index in this sorted list is used as the integer index for the Python model.

Filtering

Trials with `sampleAlg == "check"` are excluded. The `sampleSet` column (`"train"` / `"test"`) must be present and is used to split data for early stopping.

Examples

```
## Not run:
grp <- run_group_embedding_from_list(
  triplet_list = icon_triplets,
  d            = 3L,
  max_epochs   = 50000L
)

# Embedding matrix (items x dimensions)
head(grp$embedding)

# Best test loss
grp$loss

## End(Not run)
```

`setup_python_env` *Set up the Python environment for triplet embeddings*

Description

Call this function once the very first time you use the embedding pipeline. It will:

1. Check whether Miniconda/Anaconda is available on the system.
2. Create a self-contained conda environment named `envname`.
3. Install all required Python packages listed in `requirements.txt` into that environment.
4. Activate the environment for the current R session.

Usage

```
setup_python_env(envname = NULL, requirements = NULL)
```

Arguments

`envname`	Name of the conda environment to create. Defaults to `"triplet-embeddings"`. Change this only if you need to keep multiple isolated environments on the same machine.
`requirements`	Path to a `requirements.txt` file listing the Python packages to install. Defaults to the copy bundled with the package (`inst/requirements.txt`).

Details

On future R sessions you do **not** need to call this function again. Loading the package with `library()` is sufficient — the environment is detected and activated automatically at that point.

Value

The environment name, invisibly.

Python dependencies

The following packages are installed into the conda environment: `numpy`, `pandas`, `torch`, `scikit-learn`, `scipy`, and `skorch`. PyTorch is installed via conda from the pytorch channel; all other packages come from conda-forge. No pip installs are used, which ensures DLL compatibility on Windows. PyTorch is a large download (~300–800 MB depending on platform), so the first-time installation may take several minutes.

Examples

```
## Not run:
# Run once after installing the package:
setup_python_env()

# On all subsequent sessions just load the package as normal:
library(tripletTools)
results <- run_embeddings(
  input_file           = "triplets.csv",
  additional_data_file = "item_labels.csv",
  output_dir           = "embeddings_output"
)

## End(Not run)
```

---

strsplit1 *Split a string*

Description

Split a string

Usage

```
strsplit1(x, split)
```

Arguments

`x`	A character vector with one element.
`split`	What to split on.

Value

A character vector.

Examples

```
x <- "alfa,bravo,charlie,delta"
strsplit1(x, split = ",")
```

`test.model` *Test embedding model predictions.*

Description

This function generates predicted responses on a set of triplet items given an embedding of the items, and appends the model prediction as an additional column named ModPred to the triplet data file.

Usage

```
test.model(m, vdat, isemb = TRUE)
```

Arguments

`m`	An embedding of the stimuli or matrix of distances among stimuli. Rows must have names that correspond with the triplet data.
`vdat`	Data frame containing triplet data. Must include columns named Center, Left and Right, and names here must match row names of model.
`isemb`	Is model an embedding? If T (default), compute Euclidean distance matrix; otherwise just treat model as the distance matrix

Details

The returned object will be a data frame with an added field `ModPred` that contains the predicted response for the triplet given the embedding. This response will be whichever of the two choice items (Left or Right) has the smallest Euclidean distance to the target item (Center) in the embedding space.

If the triplet dataframe also includes a field labelled `Answer` that contains the true, human-generated answer for the triplet, then the model predictions can be easily converted to a proportion correct score as follows:

`mean(output$ModPred==output$Answer)`

...where `output` is the dataframe returned by the function.

Value

Returns the triplet dataframe with the column ModPred added, which contains the predicted triplet response given the embedding/distance matrix.

Examples

```
m <- data.frame(
  x=c(1,1.1,2,2.1),
  y=c(1.25,1.75,1.25,2.75))

row.names(m) <- c("cat","dog","car","boat")

m <- as.matrix(m)

tr <- data.frame(
   Center=c("cat", "car"),
```

```
    Left = c("dog", "boat"),
    Right= c("car", "dog"))

test.model(m, tr, isemb=TRUE)
```

`train_embedding` *Train a single triplet embedding model*

Description

A lower-level interface to the embedding pipeline. Use this function when you want to manage the train/test split in R rather than relying on the `sampleSet` column in your data, or when you want to train an embedding on a single subset of responses.

Usage

```
train_embedding(
  X_train,
  X_test,
  d = 5L,
  max_epochs = 50000L,
  tolerance = 1e-04,
  tol_window = 10000L,
  print_every = 100L
)
```

Arguments

`X_train`	Integer matrix of shape $n_{\text{triplets}} \times 3$. Columns must be `head`, `winner`, `loser` in that order, with zero-based integer item indices. Pass using `as.matrix(df[, c("head", "winner", "loser")])`.
`X_test`	Integer matrix in the same format as `X_train`, used for computing validation metrics and triggering early stopping.
`d`	Number of embedding dimensions. Default `5`.
`max_epochs`	Maximum number of training epochs. Default `50000`.
`tolerance`	Loss tolerance for early stopping. Default `1e-4`.
`tol_window`	Number of epochs the loss must remain within `tolerance` of the best before training halts early. Default `10000`.
`print_every`	Print a progress line every this many epochs. Default `100`. Increase to reduce console output; set to `max_epochs` to suppress mid-training output entirely.

Details

For processing all workers in a dataset at once, see `run_embeddings` and `run_embeddings_from_list`.

Value

A named list with five elements:

`embedding` Numeric matrix of shape $n_{\text{items}} \times d$ containing the learned positions. Rows correspond to items in index order.

`loss` Best test loss achieved during training.

`epoch` Epoch number at which training stopped.

`counter` Number of epochs since the last meaningful improvement at the point training stopped.

`history` Data frame with one row per epoch and columns `epoch`, `train_loss`, `test_loss`, `train_acc`, `test_acc`.

Early stopping

Training runs for up to `max_epochs` passes through the training data. It stops early if the test loss is within `tolerance` of the best observed test loss for more than `tol_window` consecutive epochs. The embedding that achieved the best test loss during training is returned, not necessarily the final-epoch embedding.

Item indices

Items are identified by zero-based integer indices. If your data uses one-based indices (as is typical in R), subtract 1 from the `head`, `winner`, and `loser` columns before passing them to this function.

Progress output

Training progress is printed to the console every `print_every` epochs, showing epoch number, train loss, test loss, train accuracy, and test accuracy. A final line is printed when training stops, labelled `[early stop]` if stopping was triggered before `max_epochs`.

Examples

```
## Not run:
triplets <- read.csv("triplets.csv")

is_train <- triplets$sampleSet == "train"
X_train  <- as.matrix(triplets[is_train,  c("head", "winner", "loser")])
X_test   <- as.matrix(triplets[!is_train, c("head", "winner", "loser")])

out <- train_embedding(X_train, X_test, d = 5L, max_epochs = 50000L)

dim(out$embedding)           # n_items x 5
cat("Best loss:", out$loss, "\n")
cat("Stopped at epoch:", out$epoch, "\n")

# Per-epoch training curve
head(out$history)
plot(out$history$epoch, out$history$test_loss, type = "l",
     xlab = "Epoch", ylab = "Test loss")

## End(Not run)
```

`z.pred.mat` *Z-score prediction matrix*

Description

This function takes a matrix of predictive accuracies as produced by `get.prediction.matrix` and returns, for each subject, a zscore indicating how predictive accuracy from a subject's own embedding relates to those from other subjects/

Usage

```
z.pred.mat(pmat)
```

Arguments

`pmat`	An embedding-to-triplet prediction matrix of the kind returned by `get.prediction.matrix`.

Details

The matrix diagonal must contain accuracies for each participant's own embedding when predicting their held-out triplet judgments. These diagonal values will be z-scored against the distribution formed by all other accuracies in the row, which reflect accuracies of *other* embeddings predicting that same participant's triplet data.

If embeddings reflect true, reliable individual differences, a participant's own embedding should predict her held-out judgments better than does another random participant from the matrix. In this case the z-scored self-prediction accuracy will be positive. If the z-score is reliably positive across the whole group of participants, this indicates reliable individual differences in representation.

Value

The predictive accuracy of each participant's embedding in predicting their own triplet decisions relative to the accuracy of other embeddings in the matrix, computed as a z-score.

Examples

```
#Random prediction matrix
pmat <- matrix(runif(25),5,5)

#Diagonal relatively high
diag(pmat) <- 1 - runif(5)/10

#Zscore diagonal relative to other entries
z.pred.mat(pmat)
```

Index